Product Requirements Document (PRD) & Technical Specification: Sistem Informasi Akademik (SIAKAD)

1. Pendahuluan (Introduction)

- **Tujuan Produk:** Menyediakan platform digital terpusat untuk mengelola administrasi akademik, mencakup manajemen data master (MDM), penjadwalan kelas, pengelolaan nilai, dan informasi tagihan mahasiswa.
- **Target Pengguna:** Admin (Pusat Kendali), Dosen (Tenaga Pendidik), dan Mahasiswa (Pengguna Akhir).

2. Rekomendasi Tech Stack

- **Backend:** Laravel (MVC Architecture, Eloquent ORM, Database Transactions).
- **Frontend:** Blade Templates + Tailwind CSS + Alpine.js / Vanilla JS (untuk toggle tab & alert).
- **Database:** MySQL / MariaDB.

3. Ruang Lingkup & Fitur Utama (Core Features)

A. Modul Admin (Master Data Management)

- **Dashboard Statistik:** Metrik jumlah mahasiswa aktif, dosen, program studi, kelas (Cards & Chart).
- **Master Data Management (CRUD):** Program Studi, Mata Kuliah, Dosen, Mahasiswa.
- **Periode Akademik:** Set semester aktif (hanya 1 periode dengan `is_active = true`).
- **Kelas Perkuliahan:** Mapping kelas (Mata Kuliah + Dosen + Hari/Jam + Ruangan + Periode Aktif).
- **Keuangan:** Set nominal tagihan per mahasiswa.

B. Modul Dosen

- **Force Profile Update:** Dosen wajib mengisi biodata & foto sebelum mengakses menu lain.
- **Jadwal Mengajar:** Ditampilkan berdasarkan `dosen_id` login & `periode_akademik_id` aktif.
- **Input Nilai (Single Page App feel):**
    - Switch tab menggunakan JS (Daftar Mahasiswa vs Form Input).
    - Input Tugas, UTS, UAS. Kalkulasi backend otomatis menggunakan Transaction.

C. Modul Mahasiswa

- **Force Profile Update:** Mahasiswa wajib melengkapi biodata.
- **Dashboard Akademik:** Chart IPS, SKS kumulatif, IPK.
- **Jadwal Kuliah:** Filter by `mahasiswa_id` & `periode_akademik_id` aktif.
- **KHS & Nilai:** Riwayat nilai huruf dan angka.
    - **Biaya Kuliah:** Status tagihan administrasi.

4. Skema Database (Entity Relationship)

Gunakan relasi snake_case standar Laravel Eloquent.

1. **users**: `id`, `username`, `password`, `role` (admin/dosen/mahasiswa).
2. **periode_akademik**: `id`, `nama_periode`, `is_active` (boolean).
3. **program_studi**: `id`, `kode_prodi`, `nama_prodi`.
4. **dosen**: `id`, `user_id`, `nip`, `nama_lengkap`, `jenis_kelamin`, `telepon`, `alamat`, `foto_profil`.
5. **mahasiswa**: `id`, `user_id`, `program_studi_id`, `npm`, `nama_lengkap`, `angkatan`, `jenis_kelamin`, `alamat`, `foto_profil`.
6. **mata_kuliah**: `id`, `kode_mk`, `nama_mk`, `sks`.
7. **kelas_perkuliahan**: `id`, `periode_akademik_id`, `mata_kuliah_id`, `dosen_id`, `nama_kelas`, `hari`, `jam_mulai`, `jam_selesai`, `ruangan`.
8. **peserta_kelas**: `id`, `kelas_perkuliahan_id`, `mahasiswa_id`, `nilai_tugas`, `nilai_uts`, `nilai_uas`, `nilai_akhir`, `nilai_huruf`.
9. **biaya_kuliah**: `id`, `mahasiswa_id`, `periode_akademik_id`, `nominal_tagihan`, `status_bayar`.

5. Struktur Routing & Middleware (Security)

Terapkan route group dan middleware untuk memisahkan akses logika secara tegas:

- `Route::middleware(['auth', 'role:admin'])->prefix('admin')`
    - `/dashboard`
    - `/master-prodi`, `/master-matkul`, `/master-dosen`, `/master-mahasiswa`
    - `/periode-akademik`, `/kelas`, `/keuangan`
- `Route::middleware(['auth', 'role:dosen'])->prefix('dosen')`
    - **Middleware Tambahan:** `CheckProfileComplete` (jika profil kosong, redirect ke `/dosen/profil`).
    - `/profil` (GET/POST)
    - `/dashboard` (Tabel Jadwal Mengajar)
    - `/kelas/{id}/nilai` (GET untuk halaman input, POST untuk *batch save* nilai).
- `Route::middleware(['auth', 'role:mahasiswa'])->prefix('mahasiswa')`
    - **Middleware Tambahan:** `CheckProfileComplete`.
    - `/profil`, `/dashboard`, `/jadwal`, `/khs`, `/biaya`

6. Logika Bisnis & Formula (Business Logic)

Kalkulasi Nilai Otomatis (Modul Dosen - Controller):
Saat Dosen melakukan POST submit form nilai, backend mengeksekusi perhitungan berikut di dalam `DB::transaction()`:

1. **Nilai Akhir (NA) Formula:**
    a. `NA = (Nilai Tugas * 0.20) + (Nilai UTS * 0.30) + (Nilai UAS * 0.50)`
2. **Konversi Nilai Huruf:**
    a. `NA >= 85` → **A**
    b. `NA >= 75 && NA < 85` → **B**
    c. `NA >= 60 && NA < 75` → **C**
    d. `NA >= 50 && NA < 60` → **D**
    e. `NA < 50` → **E**

7. Wireframe & UI Logic

- **Global Layout:** Sidebar (w-64, fixed), Header (Flex, justify-between), Main Content (flex-1, p-6, bg-gray-50).
- **Form Interceptor (Profil):** Jika flag profil belum lengkap, hilangkan/disable navigasi sidebar, render full-page form card *Lengkapi Data Diri*.
- **Input Nilai UI (JavaScript):**
    - `<div id="tab-peserta">...</div>`
    - `<div id="tab-form-nilai" class="hidden">...</div>`
    - Gunakan Vanilla JS `document.getElementById().classList.toggle('hidden')` untuk memindahkan *view* tanpa perlu me-refresh halaman (menduplikasi perilaku di video).

8. Alur Pengguna (User Flow)

1. **Admin Set Jadwal:** Admin login → Set Periode Aktif → Buat Kelas & Assign Dosen.
2. **Dosen Isi Nilai:** Dosen login → Lengkapi Profil (1x) → Klik Kelas → Switch Tab ke Input Nilai → Masukkan angka → Save. (Backend menghitung ke nilai huruf).
3. **Mahasiswa Pantau:** Mahasiswa login → Lengkapi Profil (1x) → Lihat Jadwal terfilter → Cek nilai yang sudah diinput dosen di menu KHS.
1.